EEG pretraining mini-experiments

A plain-language guide to what we are testing and why

Pavan Kalyan Tankala — v1.1, May 2026
Companion document to `experiments/exp03_eeg_pretraining/mini_experiments.md` and `brain/cortico-ssl-hypothesis.typ`.
Updated to incorporate DeeperBrain (Wang et al., Dec 2025).

Who this is for. Anyone — colleagues, advisors, collaborators — who wants to understand the EEG foundation-model work without reading the technical hypothesis document end-to-end. The text avoids jargon where possible and defines it where it cannot be avoided. Each mini-experiment is described in three short pieces: the question being asked, why the answer matters for the larger project, and how we plan to decide when one option beats another. Concrete details (compute budgets, hyperparameters, code references) live in the per-experiment `README.md` files alongside this document.

Contents

1 What we are trying to build ... 3
1.1 EEG in one paragraph ... 3
1.2 What is a foundation model and what is self-supervised learning ... 3
1.3 The specific design we want to test ... 3
2 Why mini-experiments at all ... 4
2.1 The cost of being wrong is not symmetric ... 4
2.2 Why a list of sixteen short experiments rather than one long one ... 4
3 The seven things that make EEG hard ... 5
4 The four design axes (and the choice we want to test) ... 6
4.1 Front-end ... 6
4.2 Backbone ... 6
4.3 Bottleneck ... 6
4.4 Self-supervised framework ... 6
5 The full list of mini-experiments ... 7
5.1 How the experiments depend on each other ... 8
6 The sixteen experiments, in detail ... 10
6.1 01 — Sanity baselines ... 10
6.2 02 — Front-end choice ... 10
6.3 03 — Backbone choice ... 11
6.4 04 — Self-supervised framework ... 12
6.5 05 — Multi-rate handling ... 12
6.6 06 — Reconstruction loss ... 13
6.7 07 — Phase handling ... 13
6.8 08 — Denoising target ... 14
6.9 09 — Multi-condition input ... 14
6.10 10 — Masking strategy ... 15
6.11 11 — Bottleneck (continuous vs FSQ) ... 15
6.12 12 — Quick-wins consolidation ... 16
6.13 13 — Adversarial source-dataset probe ... 17
6.14 14 — Context-length scaling ... 17
6.15 15 — Loss weights and curriculum ... 18
6.16 16 — Neurodynamics statistics prediction ... 19
7 How every experiment is evaluated ... 21
7.1 Frozen probing (the primary protocol) ... 21
7.2 End-to-end fine-tuning (the secondary protocol) ... 21
7.3 Label-free monitors (run every checkpoint) ... 21
7.4 Matched-noise twin ... 22
8 What is deliberately not in scope ... 23
9 Glossary ... 24
10 Cross-references ... 27

1 What we are trying to build

1.1 EEG in one paragraph

EEG (electroencephalography) is the recording of voltage fluctuations on the scalp produced by the collective activity of millions of cortical neurons. A typical recording uses 19–256 small metal electrodes placed on the scalp; each electrode produces one continuous waveform sampled at between 100 and 2000 times per second. The waveforms are noisy, in the range of microvolts, and dominated by a small number of standard rhythms — slow waves around 1–4 Hz called **delta**, drowsy 4–8 Hz waves called **theta**, the resting 8–12 Hz **alpha** rhythm, faster 12–30 Hz **beta** activity associated with focused attention, and 30–80 Hz **gamma** activity tied to perception and memory. EEG is attractive as a recording modality because it is non-invasive, inexpensive, and has very high time resolution; it is hard to use because the signal is small, contaminated by other electrical activity from the body (eye blinks, jaw movement, heartbeat), and varies a lot from person to person and between recording sessions.

1.2 What is a foundation model and what is self-supervised learning

A **foundation model** is a single large neural network trained once on a broad corpus of data and then reused — usually with a small amount of extra training — for many different downstream tasks. The most familiar example is a large language model: it is trained on a wide swath of text and is then adapted to question-answering, translation, summarisation, and so on. The hope, when the same idea is applied to EEG, is that a single encoder pretrained on a broad mix of EEG recordings can be adapted to clinical event detection, sleep staging, brain-computer-interface decoding, and other tasks without having to collect a large labelled dataset for each one.

Self-supervised learning is the trick that makes pretraining at this scale possible. Rather than using human-provided labels, the network is given a task that it can construct from the raw data alone. The canonical example is **masked reconstruction**: we hide a random fraction of the input from the model and ask it to predict what was hidden, given the surrounding context. To do this well, the network has to learn statistically useful features of the data — features that, with luck, also turn out to be useful for the downstream tasks we eventually care about. The advantage is that we do not need labels; the disadvantage is that we are choosing the prediction task on theoretical grounds rather than from real downstream success, which means the choice of pretraining recipe is consequential and easy to get wrong.

1.3 The specific design we want to test

The full proposed design is laid out in `brain/cortico-ssl-hypothesis.typ`. In one paragraph: we want to train a single neural network on roughly 60 000 hours of single-channel EEG, with no human labels, where each `(channel, recording, window)` triple is treated as one independent training example. The network should be robust to the noise in EEG, should preserve phase information (the timing structure of oscillations), should be able to operate at any sampling rate from 200 Hz up to 2000 Hz, and should be trained end-to-end in a single optimisation run rather than in two stages. The proposed architecture is a **bidirectional Mamba-2** encoder with a parameter-light front-end (a SincNet bandpass filterbank followed by a frozen wavelet scattering transform) and a finite-scalar-quantised bottleneck — design choices that we will explain when each becomes relevant in the document.

The reason this document exists is not to defend that design — that is what the hypothesis document is for. It is to describe the **small experiments** we plan to run before committing the compute for the full training run, in case any individual design choice is wrong.

2 Why mini-experiments at all

2.1 The cost of being wrong is not symmetric

Pretraining a foundation model is expensive. The proposed run uses multiple H100 GPUs over several days and generates several terabytes of intermediate state. The cost of finding out, after that run is finished, that one of the design choices was wrong is therefore very high — both in compute and in calendar time. The cost of finding out the same thing in a small experiment that takes one GPU-day is much lower.

Worse: pretraining at scale fails **silently**. The training loss curve will look healthy even when the underlying setup is broken. The Hugging Face **Smol Training Playbook** describes a one-trillion-token language model run that had to be restarted because every GPU in a tensor-parallel group had been initialised with the same random seed; the loss curves were perfect, the downstream evaluation revealed the bug. That happened to a tier-one lab who had derisked everything else.

The standard response, used by every modern foundation-model project we are aware of, is to spend roughly the same amount of compute on small ablation experiments as on the headline run, and to gate the headline run on the small experiments' results. **That** is what this document describes.

2.2 Why a list of sixteen short experiments rather than one long one

Each mini-experiment has been scoped to a single design decision. We hold every other axis fixed at a sensible default while varying one axis. This is the only way to get an honest read: if we change two things at once and the model improves, we cannot tell which change was responsible. For the same reason, every individual experiment also runs a **matched-noise twin**: each candidate design is also trained on a Gaussian-noise version of the data with the same statistics. A design that improves on EEG and **also** improves on Gaussian noise is just adding capacity rather than learning something specific to the signal, and we discount the improvement.

Sixteen experiments is the smallest set that covers every load-bearing choice in the proposed design. Twelve of them ablate one of the four core design axes (front-end, backbone, bottleneck, self-supervised framework) or one of the auxiliary choices (loss function, masking strategy, target signal, etc.). Four more (numbers 13–16) harden the configuration after the core stack is settled, by testing the robustness of the loss weights, the necessity of the training curriculum, the value of an explicit anti-fingerprint adversary, and the value of a **predict-the-dynamical-statistic** auxiliary head added on the basis of the recent DeeperBrain paper.

3 The seven things that make EEG hard

The phrase "EEG is low signal-to-noise" gets used a lot but it collapses several genuinely different problems into a single phrase. Most of the design choices in the proposed model exist to attack one or two of these problems specifically, so it is useful to enumerate them up front. We will refer back to these by their P-numbers throughout the rest of the document.

#	**Problem**	**What it means in plain words**
P1	Big artifacts swamp small signals	An eye blink is roughly 100 microvolts. A cognitive signal of interest like the P300 is around 5 microvolts. A loss function that simply sums the squared error treats the blink as 400 times more important than the cognition. Without care, the network learns to reconstruct blinks rather than thoughts.
P2	Different recordings have very different scales	One person's electrode is well-attached and produces a small, clean signal. The next person's electrode has poor contact and produces a signal ten times larger. The model sees these as different planets even though the underlying brain activity is comparable.
P3	Background rhythms drown out task-relevant transients	Most of the energy in a 1-second clip from a back-of-the-head electrode is the alpha rhythm. A discrete-tokenisation scheme, if not carefully designed, will spend its entire vocabulary on alpha and have nothing left over for the brief, weak, task-relevant events.
P4	The most informative band is the most contaminated	The high-gamma band (30–80 Hz), where lexical and cognitive information lives, is exactly where muscle activity also lives. We need both to be processed correctly, but the muscle contamination is large and the cognitive signal is small. Plain loss functions weighted by amplitude under-prioritise this regime.
P5	Artifacts cannot be separated by frequency alone	Eye movements, muscle activity, and heartbeat each overlap in frequency with the genuine neural signal. Bandpass filtering does not separate them; only a denoising algorithm — classical or learned — does.
P6	It is easier to learn the recording rig than the brain	Different EEG datasets are recorded with different amplifiers, different reference electrodes, different montage conventions. The lowest-loss path for the network is to learn these rig signatures and ignore the brain underneath, because rig identity is constant within a recording while neural content is fluctuating. We have to actively prevent this.
P7	There are no natural units like words or phonemes	Speech has phonemes, text has words, vision has objects. EEG has no inherent vocabulary — there is no list of "EEG units" to predict. Discrete-token training schemes have to invent the vocabulary as they go, which is harder than it sounds.

The proposed design has at least one mechanism for each of these seven problems. Several of the mini-experiments below test whether that mechanism actually works.

4 The four design axes (and the choice we want to test)

Foundation-model designs are usually decomposed into four parts. We keep that decomposition because it makes the experiments easier to reason about: each mini-experiment varies one part while holding the others fixed.

4.1 Front-end

The front-end is the first piece of the network. It receives the raw EEG waveform as input and produces a sequence of **tokens** — fixed-size vectors that the rest of the network operates on. The front-end is where classical signal-processing wisdom is most useful: bandpass filters, wavelets, and the like have eighty years of theory behind them and can be embedded directly into the network.

We want to test five front-ends, including a **SincNet** layer (a standard convolutional layer where each filter is constrained to be a bandpass with two learnable cutoff frequencies, in Hertz) and a **scattering transform** (a fixed wavelet-based feature extractor with proven mathematical properties). The hypothesis is that the combination of the two — SincNet feeding into a frozen scattering transform — is the right opening move.

4.2 Backbone

The backbone is the bulk of the network: it consumes the token sequence the front-end produces and outputs a contextualised version of the same sequence. This is where most of the parameters live.

The proposed backbone is **bidirectional Mamba-2**, a recent architecture in the **state-space model** family. State-space models are an alternative to Transformers; they process sequences in linear (rather than quadratic) time and memory, which matters at long context. The specific Mamba-2 variant has an **input-dependent gate** that lets it suppress noisy regions of the input — a useful property for EEG. We want to test it against the standard Transformer and against an LRU (a related state-space-like architecture with phase-tracking properties).

4.3 Bottleneck

The bottleneck is an optional compression layer between the encoder and decoder. It can be **continuous** (the network outputs vectors of real numbers) or **discrete** (the network outputs a token from a fixed vocabulary, like words in a language model). Discrete bottlenecks are appealing for foundation models because they create a clean interface for downstream uses (you can attach a language model to a discrete EEG token stream and treat it like text), but they are hard to train without a two-stage pipeline.

We want to test a **Finite Scalar Quantization** (FSQ) bottleneck, which is the only quantisation scheme that admits true single-stage training: the codebook is a fixed grid rather than a learned dictionary, so it cannot collapse during joint training.

4.4 Self-supervised framework

The self-supervised framework is the **recipe** — what we hide from the network, what we ask it to predict, and how the loss is computed. The proposed framework is a **masked autoencoder** with a denoised target: the network sees a partly-hidden noisy EEG window, and is asked to predict a cleaned version. We want to compare it against four alternatives — a standard masked autoencoder with raw target, a siamese decorrelation framework called VICReg, a time-frequency contrastive framework called TF-C, and a diffusion-style framework called EEGDM — to see which produces the best representations under our constraints.

5 The full list of mini-experiments

There are sixteen experiments in total, grouped into four blocks. We present them in the order in which they would naturally run.

#	Topic	Question	Days
01	Sanity baselines	Does the training and evaluation pipeline behave correctly on toy inputs before we run anything real?	0.5
02	Front-end choice	How should the raw EEG be turned into the token sequence the rest of the network sees?	2
03	Backbone choice	Which architecture (Transformer / Mamba-2 / LRU / hybrid) should sit at the centre of the network?	3
04	Self-supervised framework	Which scratch SSL recipe (masked autoencoder, VICReg, TF-C, diffusion) gives the best low-noise representation?	3
05	Multi-rate handling	How should we accommodate recordings sampled at different rates (200 Hz to 2000 Hz)?	2
06	Reconstruction loss	Which time-domain loss (L2, L1, Huber, Barron, Itakura-Saito) handles the heavy-tailed EEG amplitude distribution best?	1.5
07	Phase handling	Does an explicit phase-aware loss term materially improve representations over a magnitude-only loss?	2
08	Denoising target	Which offline cleaning method (bandpass / ICA / PCA / wavelet / IC-U-Net) gives the best target signal for the masked autoencoder to predict?	2
09	Multi-condition input	Does mixing synthetic eye-blink and muscle noise into the training input (while keeping the target clean) make the encoder more robust?	1.5
10	Masking strategy	Within a masked autoencoder, does amplitude-aware or block-structured masking beat plain random masking?	1
11	Bottleneck	Can the FSQ discrete bottleneck be trained jointly with the SSL objective in a single stage, and does it beat the continuous baseline?	2
12	Quick-wins consolidation	Do the proposed “free wins” (Snake activations, BlurPool anti-aliasing, VICReg auxiliary regulariser) survive a strict ablation when stacked together with the prior winners?	1.5
13	Adversarial dataset probe	Does adding an explicit gradient-reversal head that penalises the encoder for being able to predict the source dataset reduce rig fingerprinting?	1.5

#	Topic	Question	Days
14	Context-length scaling	Does the long-context capability that justifies Mamba-2 over a Transformer translate into better representations? At what window length?	3
15	Loss weights and curriculum	Are the proposed loss-weight values robust to small perturbations, and does the three-stage training curriculum genuinely help?	3
16	Neurodynamics statistics prediction	Does adding a small head that predicts macroscopic dynamical statistics (band power, cross-frequency coupling, sample entropy) directly from the encoder representation improve frozen-probing performance? (DeeperBrain-style.)	1.5

The first twelve are the **core stack**, working through the four design axes plus the loss heads, multi-rate handling, masking, and target choice. The next three (13–15) are **hardening experiments** added on the basis of the cortico-ssl-hypothesis: each of them tests a specific claim that is made in the hypothesis but not directly checked by experiments 01–12. The sixteenth was added on the basis of the DeeperBrain paper (arXiv 2601.06134, Dec 2025), which demonstrates that a "predict-the-statistic" auxiliary head produces dramatically better frozen-probing representations on the same backbone — see §6.16 below for what this means and §7 for the related shift in how we evaluate every experiment.

5.1 How the experiments depend on each other

```
                        01 sanity baselines
                        (gate for everything else)
                                |
        ┌───────────────────────┼───────────────────────┐
        ▼                       ▼                       ▼
   02 frontend            03 backbone          04 SSL framework
        |                       |                       |
        └──────────┬────────────┘                       |
                   ▼                                    ▼
              05 multirate          ┌──────┬──────┬──────┬──────┐
                                    ▼      ▼      ▼      ▼      ▼
                                 06 recon  08 denoised  10 mask  11 bottleneck
                                    loss      target    strategy   FSQ vs cont
                                     |         |
                                     ▼         ▼
                                 07 phase   09 multi-cond
                                  handling     input
                                     |
                                     ▼
                                  12 quick wins
                                  (final stack)
                                     |
      ┌──────────────────────┬──────┴──────┬──────────────────────────┐
      ▼                      ▼             ▼                          ▼
14 context-length scaling  13 adversarial  16 NSP auxiliary head   15 loss weights +
curriculum
                              probe        (DeeperBrain-style)
```

The graph is advisory rather than strict. Pairs of experiments that do not depend on each other can run in parallel on a multi-GPU node; 01, however, must finish before any other experiment starts, because all later experiments inherit the eval pipeline that 01 validates.

6 The sixteen experiments, in detail

For each experiment we give:

- **the question** it asks, in one line;
- **why it matters**, in one short paragraph; the relevant problem numbers (P1–P7) are noted in brackets;
- **what we will try** — a brief description of the variants;
- **how we will pick a winner** — the decision rule.

6.1 01 — Sanity baselines

> **Question.** Does the training and evaluation pipeline behave correctly on toy inputs before we run anything real?

Why it matters. Pretraining at scale fails silently. A miscalibrated trainer or evaluation pipeline produces beautifully declining loss curves on top of a fundamentally broken setup, and no later experiment can be trusted until the floor is verified. The cheapest way to catch this is a small set of "if-the-code-is-correct-this-must-pass" sanity checks. Karpathy's first rule of training neural networks is that "a fast and furious approach does not work and only leads to suffering"; this experiment is the slow, deliberate alternative.

What we will try. Five checks.

1. **Loss-at-init.** For each loss function being considered, we compare the loss value at the very start of training against the analytically expected value. A 20 %-or-larger gap indicates a bug in normalisation, shape handling, or vocabulary size.
2. **Input-independent baseline.** Replace the network's input with all zeros and train. The loss must **not** decrease meaningfully; if it does, the network is predicting the target from positional embeddings or batch composition, not from the input.
3. **One-batch overfit.** Train on four examples for a thousand steps; the loss must drive nearly to zero. If it does not, the model is too small or there is a shape-handling bug somewhere in the masking pipeline.
4. **Random-init linear-probe floor.** Run the standard evaluation suite on a freshly-initialised, never-pretrained encoder. The result is the floor against which every later experiment is compared.
5. **Shape-and-mask audit.** Print the tensor shape at every module boundary in the pipeline. A common silent bug is a `view` instead of a `transpose` somewhere in the masking code that lets the model see information from other batch entries.

How we pick a winner. This is not a winner-picking experiment; every check must pass. If any one fails, the bug is fixed and **all** checks are re-run from scratch (fixing one bug can introduce another). Until every check passes, no other experiment runs.

6.2 02 — Front-end choice

> **Question.** How should the raw EEG be turned into the token sequence the rest of the network sees? [P3, P4]

Why it matters. The front-end is the first layer the gradient signal sees. A bad one sets a ceiling on every later layer's representational quality, because no downstream layer can recover information that the front-end has already discarded. Three EEG-specific properties make this choice load-bearing: phase preservation (whether the network keeps the timing of oscillations or only their amplitude), anti-aliasing (whether the strided-convolution stack folds high frequencies back into the passband), and inductive bias (whether the network has a head start from classical signal-processing theory).

What we will try. Five front-ends.

Code	Variant	What is special about it
F0	Vanilla strided convolutions	The dumbest reasonable baseline; the floor against which everything else is compared.
F1	F0 plus Snake activations and BlurPool	Two "free wins" added: a periodic non-linearity matching EEG oscillations, and an anti-aliasing low-pass before each strided downsampling step.
F2	SincNet bandpass filterbank	Each filter is a constrained bandpass with two learnable cutoff frequencies, expressed in Hertz. Roughly 160 trainable parameters in total.
F3	Frozen Kymatio wavelet scattering	A fixed wavelet-based feature extractor with provable Lipschitz stability — small input perturbations produce small output perturbations.
F4	Complex Gabor filterbank	A bank of complex-valued bandpass filters; the real and imaginary parts together encode amplitude and phase explicitly.

How we pick a winner. On the standard evaluation suite (described in §7), strict win means the variant beats F0 by at least two percentage points on the TUEV multi-class clinical event benchmark with non-overlapping confidence intervals, **and** the matched-noise twin shows no equivalent improvement. We additionally require that the variant does not lose on a phase-locking-value reconstruction test by more than ten per cent.

6.3 03 — Backbone choice

Question. Which architecture should sit at the centre of the network and process the token sequence? [P5]

Why it matters. The backbone determines four things that downstream layers cannot recover: the effective context length the network can handle, whether phase information is tracked, whether artifact regions can be actively suppressed, and how stable training is. Recent EEG foundation models have made all four choices: EEGM2 chose Mamba-2; FEMBA chose bidirectional Mamba-1; no published EEG foundation model has used the LRU architecture. We do an iso-FLOP comparison.

What we will try. Four backbones at matched compute.

Code	Variant	Properties
B0	Bidirectional Transformer with FlashAttention-2	Standard architecture; quadratic time and memory in sequence length; no native rate handling.
B1	Bidirectional Mamba-2	Linear time and memory; input-dependent gate gives "noise gating"; no explicit phase tracking in state.
B2	Bidirectional LRU with complex eigenvalues	Linear time; complex eigenvalues encode phase rotation in state; no input selectivity for noise gating.
B3	Mamba-2 / local-attention hybrid	Five Mamba layers plus one local-window attention layer, à la Zamba-2.

How we pick a winner. The standard rule from §7. Two extra constraints: every variant must run a 60 000-sample sequence in under 60 seconds per training step on one H100 (this is a feasibility cliff the Transformer is expected to fail), and every variant must complete all five seeds without any NaN losses.

6.4 04 — Self-supervised framework

Question. Among the SSL recipes that are honestly compatible with scratch single-stage training (no teacher network, no momentum encoder, no two-stage tokeniser), which one produces the best low-noise EEG representation? [P5]

Why it matters. Most published SSL recipes for EEG sneakily depend on a teacher network or an EMA momentum encoder. When the teacher is removed and the recipe is run honestly from scratch, many of those methods lose their published numbers. The set of recipes that are **genuinely** single-encoder, scratch, and no-momentum is small; within that set, the choice substantially determines whether the model learns to reconstruct noise or signal, whether collapse is prevented architecturally, and whether phase is preserved by design.

What we will try. Six recipes.

Code	Recipe	One-line description
S0	Masked autoencoder with raw target	Hide a fraction of the input; predict the missing samples directly. The standard baseline.
S1	Masked autoencoder with denoised target	As S0, but the target is an offline-cleaned version of the input. The model is forced to predict the clean signal from the noisy one.
S2	VICReg	Two augmented views of the same window; loss balances invariance, variance, and covariance.
S3	TF-C with shared encoder	Two views — the raw signal and its complex STFT — passed through the same encoder; contrastive loss across views.
S4	EEGDM-style score-matching	Train as the conditioning network of a small diffusion denoiser. Encoder is retained for downstream use.
S5	DeeperBrain-style MER + NSP	S1 plus a small linear head that, from the encoder representation of a partly-masked input, directly predicts a 9-dimensional vector of macroscopic dynamical statistics (band power, cross-frequency coupling, sample entropy). Added on the basis of the DeeperBrain paper.

How we pick a winner. The standard rule from §7, with frozen-probing as the primary metric. One extra constraint: a "predict the source dataset" probe on the encoder's pooled output must trend **down** over training. A recipe that improves TUEV accuracy but increases source-dataset probe accuracy is learning recording-rig identity, not neural content, and is disqualified. The dedicated NSP-only ablation (weight sweep, head architecture, optional phase-locking pseudo-target) lives in §6.16.

6.5 05 — Multi-rate handling

Question. How should we accommodate recordings sampled at different rates (200 Hz to 2000 Hz)?

Why it matters. EEG datasets span an order of magnitude in sampling rate. The standard response is to resample everything down to a common rate (usually 250 Hz), which discards every frequency above 125 Hz — a band that is known to carry inner-speech and lexical information. Three published alternatives, each with their own trade-offs, have not been compared head-to-head on EEG.

What we will try. Four strategies.

Code	Strategy	Trade-off
M0	Resample everything to 250 Hz	Standard. Loses all content above 125 Hz.
M1	Mamba-rate scaling	State-space model's discretisation step is set to 1/sampling_rate, making the same continuous-time model behave correctly at any rate. Zero extra parameters.
M2	Rate-specific CNN branches	One small CNN per supported rate, all converging to a shared frame rate. More parameters; production-proven for speech.
M3	SincNet plus frozen scattering	Both layers are scale-natural by construction. Multi-rate handling is automatic.

How we pick a winner. In addition to the standard rule, two metrics matter: performance at the standard rate (so a winner must not damage the common case) and performance at a held-out higher rate (so a winner must actually preserve the high-frequency content).

6.6 06 — Reconstruction loss

Question. Within a masked autoencoder, which time-domain reconstruction loss best handles EEG's heavy-tailed amplitude distribution? [P1, P2, P3]

Why it matters. EEG amplitudes are heavy-tailed. A single eye blink dominates a squared-error loss against a small cognitive signal by a factor of several hundred. The choice of loss function is the single cheapest, most direct attack on this problem. Five options, ordered roughly by sophistication:

- **L2** (squared error): the failure baseline.
- **L1** (absolute error): cheaper, slightly more robust.
- **Huber**: quadratic for small residuals, linear for large ones; one hyperparameter.
- **Barron's adaptive robust loss**: a single family that contains L2, L1, Huber, and Cauchy as special cases, with a learnable shape parameter per channel — the model itself decides how heavy-tailed each channel is.
- **Itakura-Saito divergence on the periodogram**: scale-invariant; for Gaussian processes it equals the spectral relative-entropy rate. Has never been used as a primary EEG SSL loss before.

How we pick a winner. The standard rule from §7. One extra constraint: per-batch gradient norms must remain stable; a robust loss can fail by giving zero gradient on outliers, which makes the end-of-training metrics meaningless.

6.7 07 — Phase handling

Question. Does an explicit phase-aware loss materially improve representations over a magnitude-only loss? [P3, P4, P7]

Why it matters. "Phase" is the timing structure of an oscillation: when does the alpha rhythm peak, when does the theta wave cross zero? Many cognitive signatures of EEG live in phase rather than in amplitude — the P300, the N400, theta-gamma cross-frequency coupling. A loss that only matches amplitudes (the classical multi-resolution STFT loss) systematically discards this information, and the rest of the network has to either re-learn it from amplitude or give up on phase-coded content.

The trouble with phase is that it is a **circular** variable — 2π is the same as 0 — and the standard squared error is undefined on a circle. We test four solutions.

Code	Variant	Idea
P0	Magnitude only (baseline)	No phase term.

Code	Variant	Idea
P1	Complex STFT prediction	Predict the real and imaginary parts of the short-time Fourier transform separately. Phase is captured implicitly.
P2	Sin-cos circular loss	Predict $\sin(\varphi)$ and $\cos(\varphi)$ separately; squared error is well-defined on each.
P3	Bispectral consistency on theta / gamma	The bispectrum is a measure of cross-frequency phase coupling. We require the predicted signal's bispectrum to match the target's, restricted to the cognitively relevant band.
P4	All three combined	P1 + P2 + P3.

How we pick a winner. This experiment has two parallel decisions. First, whether the variant wins on downstream clinical-event classification (the standard rule). Second, whether the variant actually preserves phase, measured by phase-locking-value reconstruction quality on held-out segments. A variant can win on the first metric but fail on the second; in that case the gain is being attributed to "regularisation" rather than to phase preservation, which is a useful diagnostic.

6.8 08 — Denoising target

Question. If we use a masked autoencoder with a **cleaned** target (S1 from experiment 04), which offline cleaning method gives the best target signal? [P5]

Why it matters. The denoised-target trick is the most direct attack on EEG's noise problem: instead of asking the model to reconstruct a noisy signal (and therefore implicitly to learn the noise structure), we ask it to reconstruct a cleaned version. The cleaned target serves as a "what we actually want the model to predict" supervisor without requiring labels. The cleaning recipe matters: an aggressive cleaner may discard subtle phase content that was actually useful, and a permissive one barely cleans at all.

What we will try. Six target signals.

Code	Cleaning method	Trade-off
T0	Raw target (the failure baseline)	No cleaning. The model has to learn noise.
T1	Bandpass 0.5–40 Hz	Cheap, deterministic, no per-recording configuration. Discards everything above 40 Hz.
T2	ICA component rejection	The clinical gold standard. Requires multi-channel input.
T3	PCA top-8 projection	Cheap; works per-channel; keeps the dominant temporal patterns.
T4	Wavelet denoising	Removes high-frequency noise spikes; per-channel.
T5	IC-U-Net	A pretrained neural denoiser. Most aggressive; adds an external dependency.

How we pick a winner. The standard rule. Two extra constraints: each cleaned target must retain at least 80 % of the spectral power in the 1–30 Hz band of the original (a too-aggressive cleaner that just smooths out everything is unhelpful); and the model trained on the cleaned target must achieve a noticeable drop in reconstruction loss versus T0 (otherwise the cleaning is doing nothing the model could not already do).

6.9 09 — Multi-condition input

Question. Does mixing synthetic eye-blink and muscle noise into the training input — while keeping the target clean — make the encoder more noise-robust? [P5]

Why it matters. This trick comes from speech: WavLM took the standard HuBERT recipe and added one change — at training time, with some probability, mix in noise from an external library at a randomly chosen signal-to-noise ratio, and ask the model to predict the **clean** target. The result was a 23 % relative improvement in noisy-speech recognition compared to plain HuBERT. The mechanism is structural: the encoder cannot memorise the noise (which is independently sampled per training step), so the only stable strategy is to learn features invariant to the noise, which by definition makes the encoder noise-robust. The same mechanism should apply to EEG, where the "noise" is well-characterised — eye blinks, muscle activity, line noise — but the trick has never been tried on EEG.

What we will try. Five variants, differing in the type of noise injected and the injection probability. The noise types are synthetic eye-blink (broadband Gaussian shaped to the right amplitude and slow shape), synthetic muscle activity (high-frequency Gaussian with a slowly varying envelope), 50/60 Hz line noise, and broadband Gaussian. The injection probability ranges from 0 (baseline) to 0.5 (the WavLM default).

How we pick a winner. The standard rule. We additionally evaluate on the TUH artifact dataset (TUAR), which has high artifact prevalence. A multi-condition-pretrained encoder should improve on TUAR more than on TUEV; otherwise the noise injection is generic regularisation rather than artifact-specific robustness.

6.10 10 — Masking strategy

Question. Within the masked-autoencoder framework, which masking strategy produces the best representation? [P1, P3]

Why it matters. The masking strategy chooses which parts of the input the model is forced to predict. For natural images, random patch masking at 75 % works because images are highly locally redundant. EEG has analogous redundancy in time, but with two specific failure modes: long stretches of stable rhythmic activity (alpha during eyes-closed rest) can be reconstructed by trivial linear interpolation between unmasked patches — the loss decreases beautifully without the network learning anything; and high-amplitude artifact regions get masked as often as signal regions, but the model's loss is then dominated by trying to reconstruct the artifact (which is by definition unpredictable).

What we will try. Five strategies.

Code	Variant	Idea
K0	Random patch mask (vanilla)	Mask 50 % of patches at random.
K1	Block masking	Mask one or two contiguous blocks totalling 50 %.
K2	Semantic-subsequence-preserving (SSP)	Choose two subsequences to **preserve**; mask everything else.
K3	Multi-block masking (I-JEPA-style)	Mask four large contiguous blocks totalling 60 %.
K4	Amplitude-aware masking (AAMP)	Preferentially mask the highest-amplitude patches, forcing the model to reconstruct them from quiet context.

How we pick a winner. The standard rule. One extra constraint: the end-of-training reconstruction loss must remain non-trivially above zero. A masking strategy that reconstructs to almost zero is solving a trivial task; the network has learned nothing.

6.11 11 — Bottleneck (continuous vs FSQ)

Question. Can the FSQ discrete bottleneck be trained jointly with the SSL objective in a single stage, and does it beat the continuous baseline? [P1, P7]

Why it matters. Every published EEG model with a discrete bottleneck trains it in two stages: tokeniser first, then the encoder is frozen and the masked-prediction transformer is trained on top. The single-stage constraint we have committed to rules this out. Within the single-stage-compatible options, **Finite Scalar Quantization** (FSQ) is the only quantiser that works: the codebook is a fixed grid rather than a learned dictionary, so it cannot collapse during joint training, and 100 % codebook utilisation is structurally guaranteed.

The honest empirical question is whether the gains often attributed to discrete bottlenecks (better cross-subject transfer, cleaner interface for downstream language models) survive single-stage joint training, or whether the continuous baseline is uniformly better.

What we will try. Three variants.

Code	Variant	Properties
Q0	Continuous (baseline)	No quantisation.
Q1	FSQ at modest size, 15 625 effective codes	Six-dimensional grid with 5 levels per dimension.
Q2	FSQ at larger size, 36 000 effective codes	Six-dimensional grid with mixed levels.

How we pick a winner. The standard rule on the standard split, **plus** a leave-one-subject-out evaluation. The well-documented pattern in the EEG-FM literature is that continuous bottlenecks win on within-subject metrics while discrete bottlenecks win on cross-subject transfer. We test whether that pattern survives the single-stage constraint.

6.12 12 — Quick-wins consolidation

Question. Do the proposed "free wins" survive a strict ablation when stacked together with the prior winners?

Why it matters. Three modifications were proposed across the research conversation as "free wins" that should be applied universally: **Snake activations** (a learnable periodic non-linearity matched to oscillatory signals), **BlurPool** (an anti-aliasing low-pass before each strided downsampling), and **VICReg as an auxiliary regulariser** (the variance-invariance-covariance loss applied to the encoder's pooled output, designed to suppress representation collapse). Every one of them looks free in isolation. The question is whether they actually compose with each other and with the choices made in experiments 02–11.

The legitimate worry is that gains from individual experiments do **not** compose: what helps in isolation can hurt in combination because the gradient signal, the loss balance, or the regularisation regime changes. This is a documented failure mode in foundation-model training and is the reason every project of this kind ends with a consolidation pass.

What we will try. The experiment runs in two phases. **Phase A** isolates each "free win" against the experiment 02–11 best configuration: W1 adds Snake, W2 adds BlurPool, W3 adds the VICReg auxiliary. **Phase B** — only run if all three Phase A variants strict-win — combines all three (W4) and asks whether the combination beats the best individual.

How we pick a winner. The standard rule with a lower bar (0.5 pp TUEV BAC) for individual quick wins. For W4, the combination must strict-win against the **best** of the three individuals, not merely against the baseline; this protects against false additivity claims.

6.13 13 — Adversarial source-dataset probe

Question. Does an explicit gradient-reversal head that penalises the encoder for being able to predict the source dataset of each window reduce rig fingerprinting (P6) without hurting downstream task performance?

Why it matters. This is one of the three hardening experiments added on the basis of the cortico-ssl-hypothesis. The hypothesis lists a "predict the source dataset" **adversarial** probe as one of seven loss heads — the cheapest defence against P6 (the pathology where the encoder learns rig identity instead of brain content). Our target is to keep the dataset-identification accuracy below 50 % on a four-dataset mix where chance is 25 %.

The earlier experiments only **monitor** this probe as a sanity check. None of them tests whether **adding** the gradient-reversal head actually helps. The risk is real: the head might be redundant (other tricks already handle P6), or it might be too aggressive and suppress useful features that happen to correlate with rig identity (sampling-rate biases, age-related alpha-peak shifts).

Gradient reversal, in one sentence: a layer that passes its input unchanged on the forward pass, but on the backward pass multiplies the gradient by $-\alpha$ before sending it into the rest of the network. The classifier learns to predict the dataset; the encoder is pushed in the **opposite** direction, so that the encoder representations become uninformative about dataset identity. This is the standard **Domain-Adversarial Neural Network** (DANN) recipe of Ganin et al.

What we will try. Five variants.

Code	Variant	What is varied
A0	No adversary (baseline)	Pure W4 from experiment 12.
A1	Linear adversary, weight 0.05	Single linear classifier; weak penalty.
A2	Linear adversary, weight 0.20	Single linear classifier; stronger penalty.
A3	MLP adversary, weight 0.10	Two-layer classifier; moderate penalty.
A4	Dataset + subject-ID adversary	Two MLP heads — one for dataset, one for individual subject.

How we pick a winner. This experiment has two parallel decisions. First, **do not lose**: the variant must be within 0.5 percentage points of the baseline on TUEV BAC. Second, **invariance achieved**: the source-dataset linear-probe accuracy at the end of training must be below 50 % to count as a strict pass. A variant must satisfy both conditions to be adopted. If no variant does, the headline run ships without the adversary and the cortico-ssl-hypothesis's H6 prediction is downgraded to a passive monitor.

6.14 14 — Context-length scaling

Question. Does the long-context capability that justifies Mamba-2 over a Transformer translate into better representations? At which pretraining window length?

Why it matters. This is the second of the three hardening experiments. The cortico-ssl-hypothesis claims that the model is rate-invariant up to 2 kHz, where a 30-second window contains 60 000 samples. A Transformer with FlashAttention-2 is intractable at this scale on a single H100 — its memory grows quadratically with sequence length. The entire argument for picking Mamba-2 over a Transformer rests on the long-context regime being (a) feasible only for the state-space family and (b) actually beneficial for representation quality.

The claim is plausible on theory grounds. EEGM2 — a recent Mamba-2 based EEG foundation model — reported that performance peaks at 30-second windows on the TUAB benchmark before plateauing.

But experiments 02–13 all use the standard 4-second window, so we have not actually verified the long-context benefit in our recipe.

What we will try. Four window lengths, each tested on both Mamba-2 and Transformer backbones.

Code	Window	# samples at 250 Hz	Notes
C0	4 s	1 000	Default; both backbones feasible.
C1	8 s	2 000	Both feasible; Transformer starts to slow.
C2	16 s	4 000	Mamba-2 still cheap; Transformer painful.
C3	30 s	7 500	Mamba-2 feasible; Transformer expected to OOM at 2 kHz eval.

Each variant is trained on the same number of total samples seen (roughly 35 million tokens), so longer-window cells use proportionally larger batches but fewer optimiser steps. This makes representation-quality comparisons fair; matching steps would unfairly favour shorter windows.

How we pick a winner. Two metrics matter. First, downstream representation quality on the standard evaluation suite, evaluated at the standard 4-second eval window across all variants — only **pretraining** window varies. Second, **Sleep-EDF** sleep-stage classification, where the AASM-standard 30-second epochs make the long-context argument concrete. The decision is whether the absolute gain from longer windows justifies the wall-clock cost; longer pretraining windows cost roughly proportionally more time.

6.15 15 — Loss weights and curriculum

Question. Are the proposed loss-weight values robust to small perturbations, and does the proposed three-stage training curriculum genuinely improve over training the full loss from step 0?

Why it matters. The proposed model has seven loss heads, each attacking a different EEG pathology, with weights $(1.0, 0.3, 1.0, 0.1, 0.05, 1.0, 0.1)$ chosen on theoretical grounds. The hypothesis itself flags weights as "the part of the recipe most likely to need tuning". Multi-objective optimisation is brittle in two ways: a single dominant head can effectively reduce the others to noise, and starting all losses at step 0 produces three known failure modes (the FSQ codes are random at step 0, so a strong code-prediction loss forces the encoder to predict noise; the bispectral loss requires a meaningful spectrum to compute against, which an untrained network does not produce; the adversarial head needs the encoder to encode dataset identity **before** it can suppress it).

The hypothesis specifies a three-stage curriculum in response: start with two losses only, then ramp the others in over a window, then turn on the full loss. This is reasonable, but never validated empirically. Experiment 15 splits the question into two phases.

What we will try.

Phase A (loss-weight sensitivity). We perturb each weight by a factor of two (up or down) while holding the others fixed at the hypothesis values. Eight cells in total — seven one-at-a-time perturbations plus one **adaptive-weight** control (GradNorm) where the weights are adjusted on the fly to keep gradient magnitudes balanced.

Phase B (curriculum). Four schedules: training without curriculum (all losses on from step 0), the hypothesis three-stage curriculum, an FSQ-only warmup (the simplest variant where only the discrete bottleneck gets ramped in), and a longer warmup (the hypothesis schedule stretched over twice as many steps).

How we pick a winner. For Phase A, we expect non-degradation rather than improvement: the hypothesis recipe should be near-optimal, and we are checking that small perturbations do not break it.

A perturbation that loses by 0.5 percentage points or more on TUEV BAC is **rejected** and indicates the recipe is brittle along that axis. For Phase B, the hypothesis curriculum (Cu1) is preferred if it beats the no-curriculum baseline by 0.5 pp on TUEV BAC, **or** if it raises end-of-training FSQ codebook utilisation from below 50 % (the structural failure mode the curriculum is designed to prevent) to above 80 %.

6.16 16 — Neurodynamics statistics prediction

Question. Does adding a small head that, from the encoder representation of a partly-masked input, **directly predicts** macroscopic dynamical statistics of the full window — relative spectral power per band, cross-frequency coupling, sample entropy — materially improve frozen-probing performance beyond the existing reconstruction-based losses?

Why it matters. This is the fourth hardening experiment, added on the basis of the DeeperBrain paper (arXiv 2601.06134, Dec 2025). DeeperBrain's central methodological argument is that there are two different ways to use a neurodynamical statistic in an SSL loss, and they produce qualitatively different representations:

- **Match the statistic.** Compute the statistic on both the predicted reconstruction and the ground-truth target, and minimise the squared error between the two. The encoder is constrained only **indirectly**: it must produce a representation whose decoded waveform exhibits the right statistic. Our experiments 06 and 07 (Itakura-Saito divergence, multi-resolution STFT, bispectral consistency) all use this framing.
- **Predict the statistic.** Attach a separate small linear head to the encoder representation that produces the statistic value as an output, with no decoder involved. The encoder is constrained **directly**: its representation has to encode the statistic linearly, because a linear head will be reading it off. This is the DeeperBrain framing.

The second is much stronger as a constraint on the encoder representation, and DeeperBrain's Table IV shows that the practical consequence is dramatic. Under frozen probing on FACED 9-class emotion, DeeperBrain reaches 50.96 % balanced accuracy while CBraMod reaches 25.84 % and LaBraM reaches 16.13 %. Under fine-tuning the same models are within 5 pp of each other. Frozen probing exposes the representation-quality difference because the encoder cannot adapt at fine-tune time — and the predict-the-statistic framing is what gets the encoder to produce a representation that survives the freeze.

The honest empirical question for our recipe is whether this extra head adds anything once we have already paid for the four spectral and phase loss heads in the cortico-ssl-hypothesis. It might be fully redundant, partially complementary, or critically additive.

What we will try. Five variants on top of the experiment 12 W4 configuration.

Code	Variant	What changes
N0	No NSP head (baseline)	Pure W4 from experiment 12.
N1	Linear NSP head, low weight	9-dimensional target (5-band power, 3 cross-frequency couplings, sample entropy); $\lambda_{\text{NSP}} = 0.3$.
N2	Linear NSP head, full weight	Same head, $\lambda_{\text{NSP}} = 1.0$. Matches DeeperBrain's reported setting.
N3	MLP NSP head, full weight	Two-layer head with hidden $\frac{D}{2}$; $\lambda_{\text{NSP}} = 1.0$.
N4	N2 plus phase-locking pseudo-target	Adds a 2-D phase-locking summary computed against a randomly-paired second channel from the same recording. Breaks strict iid-channel pretraining.

The last variant, N4, is the only place where iid-channel pretraining is explicitly tested against a multi-channel auxiliary signal. If N4 strict-wins, the iid-channel decision becomes a topic for the channel-aggregation work in `brain/experiments/pretraining-experiment/`, not a settled axis.

How we pick a winner. The primary metric is **frozen-probing** performance on TUEV (balanced accuracy), with the standard rule from §7. A variant must additionally not lose by more than 0.5 percentage points under fine-tuning — we want NSP to help, but not at the cost of fine-tune quality. We also report per-statistic correlations between the NSP head's predictions and the ground-truth statistic on a held-out set: alpha power, sample entropy, the three cross-frequency couplings. DeeperBrain reports alpha power $r \approx 0.82$, sample entropy $r \approx 0.75$, and cross-frequency coupling near zero (the latter is informative — it suggests the model is correctly refusing to overfit a high-noise statistic). We expect a similar pattern.

7 How every experiment is evaluated

Every mini-experiment uses the same evaluation suite, run after pretraining. The suite has two protocols — **frozen probing** (the primary metric we care about) and **end-to-end fine-tuning** (a secondary sanity check) — plus a small set of label-free monitors logged every checkpoint.

7.1 Frozen probing (the primary protocol)

Under frozen probing, the pretrained encoder's parameters are **not allowed to change** during downstream evaluation. A single linear classifier (or, for k-NN, no classifier at all) is trained on top of the frozen encoder's mean-pooled output. This is the canonical "linear probe" of the image-SSL literature.

We treat frozen probing as the **primary** metric for two reasons. First, it is the truer test of "did the SSL produce a useful representation": fine-tuning can compensate for a mediocre representation by re-optimising the entire encoder, but a linear classifier on a frozen encoder cannot. Second, the recent DeeperBrain paper (arXiv 2601.06134, Dec 2025) demonstrates empirically that several published EEG foundation models — LaBraM, CBraMod, CSBrain, REVE — match each other to within 2 percentage points under fine-tuning, but diverge by 10 to 35 percentage points under frozen probing. On a number of tasks the latter group of models drops to chance accuracy under frozen probing, even though their fine-tuned numbers are competitive. This pattern means fine-tuning is hiding the representational quality difference, and frozen probing is the only protocol that surfaces it. We do not want to ship a model whose representation is only good once the encoder has been re-optimised for a specific task — that is not a foundation model in the sense the term is usually used.

The three frozen-probing tasks are:

- **Linear probe on TUAB (binary).** The Temple University Hospital Abnormal EEG dataset, normal-vs-abnormal classification. Single-layer linear head on top of the frozen encoder. Metric: AUROC.
- **Linear probe on TUEV (multi-class).** The TUH Event corpus, six clinical event classes (spike-and-wave, generalised periodic discharges, periodic lateralised epileptiform discharges, eye movement artifact, generic artifact, background). Same protocol as TUAB. Metric: balanced accuracy and weighted F1. This is the headline metric for most of the experiments in §6.
- **k-NN on a 10 000-sample TUEV subset.** A 5-nearest-neighbours classifier on the frozen encoder's mean-pooled output, using cosine distance. This requires no training at all and so cannot be gamed by clever head architecture; it is particularly sensitive to representational collapse.

7.2 End-to-end fine-tuning (the secondary protocol)

Under fine-tuning, the entire encoder is unfrozen and jointly optimised with the classifier head on the downstream task. We report this for completeness — it is the field-standard protocol — but a variant that wins under fine-tuning and loses under frozen probing is not adopted. It is just a transferable initialisation, not a universal representation.

The decision rule for every experiment in §6 is therefore: a candidate must (a) win on the frozen-probe metric by the margin specified in the experiment, and (b) not hurt the fine-tune metric by more than 0.5 percentage points relative to the baseline. The first is the headline test; the second is a sanity bound.

7.3 Label-free monitors (run every checkpoint)

Four monitors that do not depend on any labels:

- **Encoder feature standard deviation.** Should be stable across training; a sharp decrease signals representational collapse.
- **Encoder feature absmax/std ratio.** Should be bounded; an unbounded ratio signals exploding individual features.
- **Encoder covariance rank.** Should remain at least half the feature dimension; collapse manifests as low rank.

- **Source-dataset linear probe.** Should **decrease** over training; a rising probe means the encoder is learning rig identity instead of brain content.

7.4 Matched-noise twin

Every cell of every experiment is also run on a **matched-noise** version of the data: Gaussian noise with the same per-window mean and variance as the original EEG. The noise twin must **not** show the same improvement as the EEG cell. If it does, the gain is being attributed to architectural priors rather than to EEG-specific structure, and we discount it. This is the protocol of Jo et al. 2024, applied at every level of the experiment hierarchy rather than only at the headline run.

8 What is deliberately not in scope

The fifteen experiments cover all four axes of the proposed design, but several other questions are deferred or excluded by prior decisions:

- **Multi-channel vs single-channel pretraining.** The choice to treat each `(channel, recording, window)` triple as an independent training example was made upstream of this folder and is being separately tested in `brain/experiments/pretraining-experiment/`.
- **Teacher–student methods, EMA momentum encoders, and two-stage tokenisers.** These are excluded by the scratch single-encoder constraint. Specifically, this rules out SALT, BYOL, DINO, data2vec, EEG2Rep, EEGPT, LaBraM, NeuroLM, and CBraMod-style training. Some of their loss-level or architectural ideas are still borrowed inside individual cells.
- **The full-scale headline pretraining run itself.** That is Phase 4 in the project methodology and only happens after these mini-experiments have settled the configuration.
- **EEG-to-text fine-tuning.** The evaluation suite here is restricted to TUAB / TUEV linear probe and k-NN, which run cheaply (under ten minutes per experiment cell). Whether the resulting encoder is good enough to feed into the ZuCo / Brennan / Chisco fine-tunes is a separate decision once we have a winner.
- **Hyperparameter search beyond learning-rate sweeps.** Width and depth scaling are reserved for the intermediate-scale validation phase. Each recipe in the mini-experiments gets a three-point learning-rate sweep at proxy scale; no exotic hyperparameter search is performed.

9 Glossary

For convenience, we collect the technical terms that appear in this document with one-sentence definitions.

Ablation. A controlled experiment that varies one design choice while holding everything else fixed.

AAMP. Amplitude-aware masked pretraining. A masking strategy that preferentially hides high-amplitude regions.

Alpha (rhythm). The 8–12 Hz EEG oscillation, prominent during relaxed wakefulness with eyes closed.

AUROC. Area under the receiver-operating-characteristic curve. A binary-classification metric between 0.5 (chance) and 1.0 (perfect).

Backbone. The bulk of the network — the architecture (Transformer, Mamba-2, etc.) that processes the token sequence the front-end produces.

Bandpass filter. A signal-processing operation that keeps only frequencies in a specified range and removes everything outside it.

Barron loss. A robust regression loss with a learnable shape parameter; subsumes squared error, absolute error, Huber, and Cauchy as special cases.

Beta. The 12–30 Hz EEG band, associated with focused attention.

Bicoherence (bispectrum). A statistical measure of cross-frequency phase coupling between three frequencies f_1, f_2, $f_1 + f_2$.

BlurPool. An anti-aliasing low-pass filter applied before strided downsampling. Prevents high frequencies from folding back into the passband.

Bottleneck. An optional compression layer between encoder and decoder. Can be continuous or discrete.

Codebook. In a discrete bottleneck, the fixed list of vectors the model can output.

Continuous bottleneck. A bottleneck that outputs vectors of real numbers without discretisation.

DANN. Domain-adversarial neural network. A training scheme that uses a gradient-reversal layer to make encoder representations uninformative about a nuisance variable like dataset or subject identity.

DeeperBrain. An EEG foundation model (Wang et al., arXiv 2601.06134, Dec 2025) that introduces volume-conduction-aware spatial encoding, neurodynamics-aware temporal encoding, and the Neurodynamics Statistics Prediction (NSP) auxiliary objective; reports dramatic frozen-probing gains over LaBraM, CBraMod, CSBrain, and REVE on a shared benchmark.

Delta (rhythm). The 1–4 Hz EEG band, prominent during deep sleep.

EEG. Electroencephalography. Recording of voltage fluctuations on the scalp produced by cortical neurons.

EMG. Electromyography. Electrical signal from muscles, often contaminating EEG recordings.

EOG. Electro-oculography. Electrical signal from eye movements, often contaminating EEG recordings.

Encoder. The part of the network that takes the input and produces a latent representation.

FSQ. Finite Scalar Quantization. A discrete bottleneck with a fixed grid (rather than a learned dictionary). Cannot collapse during training.

Front-end. The first part of the network. Maps the raw input signal into the token sequence the backbone consumes.

Frozen probing. An evaluation protocol where the pretrained encoder's parameters are not allowed to change during downstream evaluation; only a small classifier on top of the encoder's output is trained. The truer test of whether a self-supervised encoder has learned a useful representation than end-to-end fine-tuning.

Gamma. The 30–80 Hz EEG band, associated with perception and memory.

Gradient-reversal layer. A layer that passes its input unchanged on the forward pass and multiplies the gradient by $-\alpha$ on the backward pass.

Huber loss. A regression loss that is quadratic for small errors and linear for large errors.

ICA. Independent component analysis. A classical method for separating mixed signals; used in EEG to isolate and remove eye blinks, muscle activity, and heartbeat.

iid (channel framing). Independent and identically distributed. In our setting, each `(channel, recording, window)` triple is treated as an independent training example.

Itakura-Saito divergence. A scale-invariant divergence between positive functions; for Gaussian processes equals the spectral relative-entropy rate.

Kymatio. A Python library for wavelet scattering transforms.

LRU. Linear Recurrent Unit. A linear-time architecture related to state-space models, with complex eigenvalues that explicitly encode phase rotation.

Mamba / Mamba-2. A modern state-space-model architecture; linear in sequence length; has an input-dependent gate.

Masked autoencoder (MAE). A self-supervised recipe that hides part of the input and asks the network to predict the hidden part.

Masking strategy. The rule that decides which patches of the input to hide during MAE training.

MR-STFT. Multi-resolution short-time Fourier transform. A loss that compares predicted and target spectra at multiple time resolutions.

NSP (Neurodynamics Statistics Prediction). A self-supervised loss introduced by DeeperBrain in which a small head reads off macroscopic dynamical statistics — band power, cross-frequency coupling, sample entropy, optionally phase-locking value — directly from the encoder representation, without going through a decoder. Distinct from reconstruction-based losses that compute the same statistic on a decoded waveform and match it to ground truth.

Phase. The timing of an oscillation; the position within one cycle at a given moment.

Phase-locking value (PLV). A measure between 0 and 1 of how reliably the phase of one signal aligns with the phase of another.

PLV. See phase-locking value.

Self-supervised learning (SSL). Learning a representation from unlabelled data using a task constructed from the data itself.

SincNet. A convolutional layer where each filter is constrained to be a bandpass with two learnable cutoff frequencies.

State-space model. A class of sequence-processing architectures that are linear in sequence length and based on a continuous-time dynamical system.

STFT. Short-time Fourier transform. The standard time-frequency decomposition of a signal.

Theta (rhythm). The 4–8 Hz EEG band, associated with drowsiness and memory encoding.

Token. A fixed-size vector. The unit on which the backbone operates.

Transformer. The standard attention-based architecture; quadratic in sequence length.

TUAB. TUH Abnormal EEG corpus. A standard benchmark for normal-vs-abnormal classification.

TUEG. TUH EEG corpus. A large unlabelled clinical EEG dataset used for pretraining.

TUEV. TUH EEG Events corpus. A standard benchmark with six clinical event classes.

VICReg. Variance-Invariance-Covariance Regularisation. A self-supervised loss based on three penalties (invariance to augmentations, variance preservation, decorrelation of latent dimensions).

Wavelet. A short, localised, oscillatory function used as a basis for time-frequency analysis.

Wavelet scattering. A wavelet-based feature extractor with provable mathematical stability properties; the filters are fixed rather than learned.

10 Cross-references

For further detail on any individual experiment, see the corresponding `README.md` in the `mini_experiments/NN_topic/` folder. The full proposed design is in `brain/cortico-ssl-hypothesis.typ`. The general methodology — eval discipline, monitoring, the five-phase plan — is in `experiments/exp03_eeg_pretraining/methodology.md`. The list of experiments and the dependency graph in machine-readable form is in `experiments/exp03_eeg_pretraining/mini_experiments.md`.